

Project Report

Resume Screening and ATS Scoring Web App

Prepared by: Prapti Manoj Kambli

Tools used: Python, Flask, scikit-learn

Date: July 2026

Intern ID: 415

What this project is about:

This is a Flask web app where you upload a resume (PDF) and it tells you what job category the resume fits into, gives it an ATS-style score, and lists which skills it found and which ones are missing for that category. Basically, it's a follow-up to my earlier Streamlit resume classifier, but this time I wanted it to actually be useful for someone checking their own resume, not just a demo that spits out a category label.

The core of it is a text classification model, same idea as before: clean the resume text, turn it into TF-IDF vectors, feed it to a classifier. What's new here is the skills.py module, which does the ATS scoring and skill-gap part on top of the prediction.

Dataset:

I used a different dataset this time, Resume.csv, from [Kaggle.com](https://www.kaggle.com/datasets/praptikambli/resume), which has 2,484 resumes across 24 categories (HR, Information Technology, Teacher, Advocate, Business Development, Healthcare, and so on). Each row has the resume text, an HTML version of it, and the category label. No missing values, so cleaning was mostly about the text itself rather than fixing the dataframe.

The categories aren't evenly distributed. Information Technology and Business Development have around 120 resumes each, while a few categories like Automobile (36) and BPO (22) have a lot fewer. This ends up mattering later, because the model is noticeably worse on the categories with fewer examples.

Cleaning the text

Same general approach as the last project: lowercase everything, strip out URLs, email addresses, and punctuation, then collapse extra whitespace. Nothing fancy, just enough so the vectorizer isn't picking up noise like email domains or stray symbols as if they were meaningful words.

Turning text into features

I used TF-IDF again, but tuned it a bit differently this time: `max_features=7000`, `ngram_range=(1,2)` so it picks up both single words and two-word phrases, `min_df=2` to drop words that only show up once in the whole dataset, and `max_df=0.95` to ignore words that appear in almost every resume (things like “experience” or “team” that don't tell you much about category). This gave a 2,484 x 7,000 feature matrix.

The data was split 80/20 (1,987 training resumes, 497 test resumes), stratified by category so the split doesn't accidentally drop small categories entirely.

Trying a few models

I trained three different classifiers on the same train/test split to see which handled the data best:

Model	Test accuracy
Logistic Regression	65.6%
Linear SVM	72.8%
Multinomial Naive Bayes	56.1%

Linear SVM came out ahead by a decent margin, so that's the one I saved for the app. Logistic Regression wasn't far behind, and Naive Bayes was clearly the weakest of the three here, which tracks since it tends to struggle when a lot of words overlap across categories the way resume vocabulary does.

Being honest about the accuracy

72.8% is fine but it's not great, and I spent a while trying to figure out why it wasn't higher. I checked for the obvious stuff first: no nulls in the dataset, the label encoding looked correct, the train/test split had all 24 categories represented in both sets, and spot-checking individual predictions (like the HR example) came back correct. So it's not a bug in the pipeline as far as I can tell.

Looking at the classification report, the model does well on categories with more training examples (Construction, Digital Media, HR all sit around 75-85% F1) and noticeably worse on smaller or more overlapping categories — categories like Consultant and Aviation had a lot of confusion with other similar-sounding categories in the confusion matrix. That lines up with the class imbalance in the dataset, and honestly with how similar some resume categories are to begin with (a Business Development resume and a Sales resume aren't going to look that different in plain text).

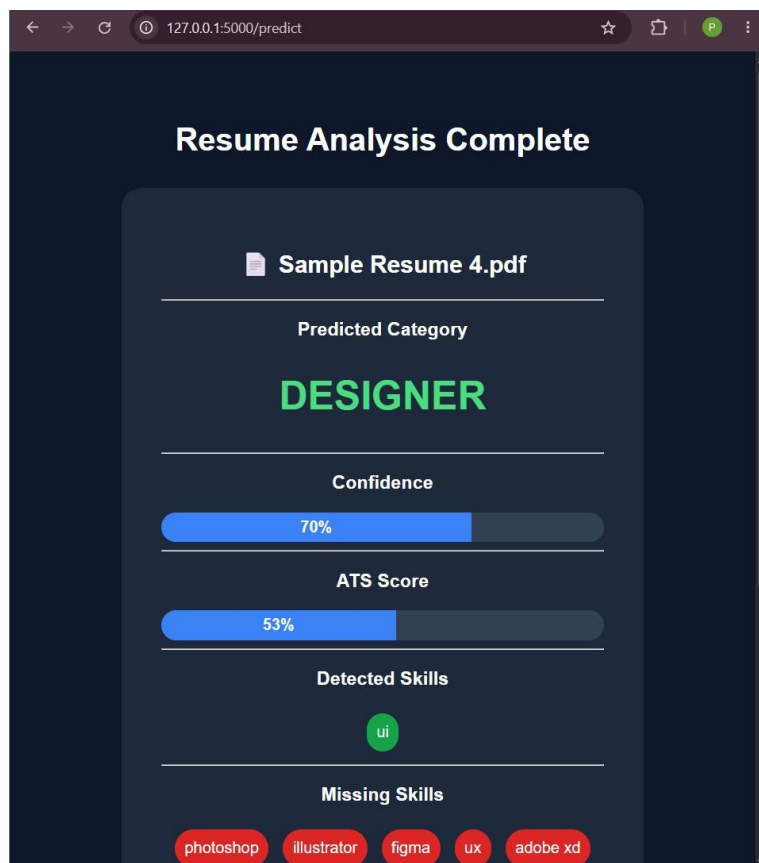
The Flask app:

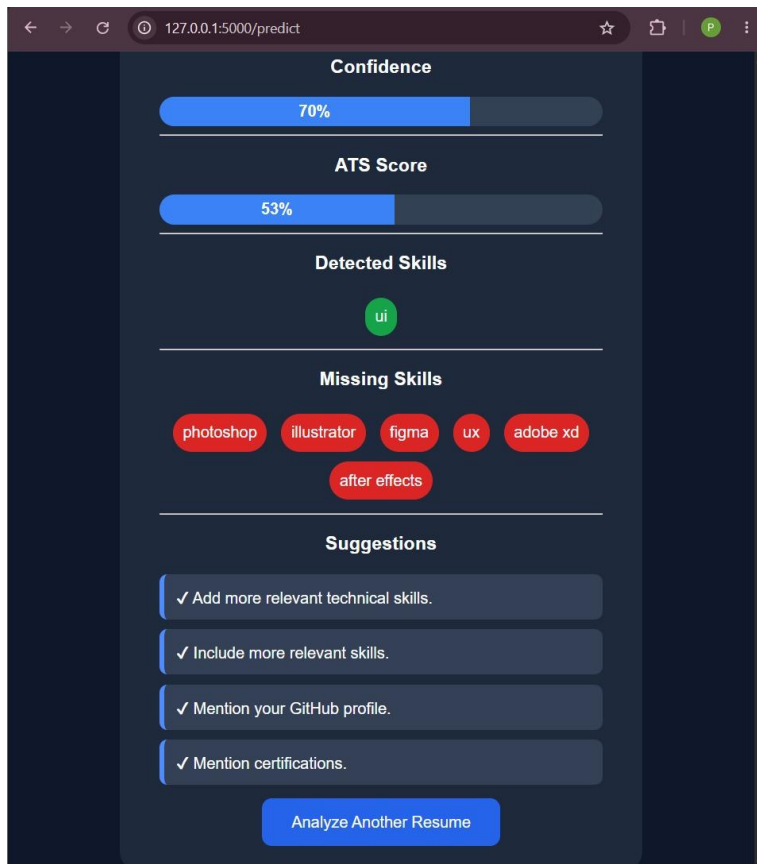
app.py handles the actual web app. There's a home route that just renders the upload page, and a /predict route that does the real work when someone submits a resume:

- Saves the uploaded PDF to the uploads folder.
- Pulls the text out using pdfplumber, page by page.
- Cleans it with the same clean_resume function used during training.
- Vectorizes it with the saved TF-IDF vectorizer and predicts a category with the saved SVM model.
- Passes that category into skills.py to get the detected skills, ATS score, missing skills, and a few suggestions.
- Also calculates a rough confidence percentage from the model's decision function, capped between 70 and 97 so it doesn't show something silly like 3% or 100%.
- Renders all of it on a results page.

Sample output

Below is what the results page actually looks like when a resume is uploaded and processed — in this case, Sample Resume 4.pdf, predicted as Designer with a 70% confidence and a 53% ATS score. There are 2-3 more Resumes given for testing, but you can use any resume of your choice as well (IN PDF FORMAT ONLY)





The detected skill was just “ui”, while the missing skills list (photoshop, illustrator, figma, ux, adobe xd, after effects) points out what the resume didn't mention. The suggestions at the bottom are the generic ones from `get_suggestions()` — in this case all four fired, meaning the resume didn't mention GitHub, certifications, and had fewer than 5 detected skills.

How the ATS scoring works (skills.py)

This part isn't machine learning, it's just a rules-based system I wrote on top of the prediction. There's a `skills_database` dictionary with a list of relevant keywords per category — for example HR looks for things like recruitment, payroll, onboarding, and compliance, while Information Technology looks for python, sql, git, flask, and so on. Categories that don't have a specific list defined yet fall back to a default set of generic skills (communication, teamwork, leadership, etc.), which I added so the app doesn't break on categories I haven't filled in keyword lists for yet.

Once the category is predicted, `extract_skills()` just checks which of that category's keywords show up in the cleaned resume text. `calculate_ats()` turns that into a score out of 100 (starting at a base of 45 and adding up to 55 more points depending on how many of the expected skills were found). `missing_skills()` is just the keywords that weren't found, so the user can see what's missing. `get_suggestions()` adds a few generic tips — things like mentioning a GitHub profile,

adding project experience, or listing certifications, if those words don't show up anywhere in the resume.

It's a simple keyword-matching approach, not a real ATS parser, but it gives the user something more actionable than just a category label.

Limitations

- 72.8% accuracy means roughly 1 in 4 resumes gets misclassified, which then throws off the ATS score and skill list too, since both depend on the predicted category being right.
- Smaller categories like BPO and Automobile don't have enough training examples for the model to learn them well.
- The skills database is keyword-based, so it can miss a skill if it's phrased differently than expected, or count a skill as present just because the word appears somewhere out of context.
- Only PDF uploads are handled in this version, even though earlier the app had DOCX/TXT support too.
- The confidence percentage is a rough heuristic based on the SVM decision function, not a calibrated probability.

What I'd improve next

- Try balancing the dataset (oversampling the smaller categories) to see if it helps the weaker categories specifically.
- Fill in proper skill lists for the categories still using the default generic list.
- Add DOCX and TXT support back in, since resumes come in all formats.
- Show the predicted category's confidence alongside the top 2-3 alternatives, instead of just one label.
- Look into whether a transformer-based model would actually do meaningfully better here, or if the ceiling is more about how similar some categories are to each other.

Tech stack

- Python
- Flask (web app)
- scikit-learn (TfidfVectorizer, LabelEncoder, LogisticRegression, LinearSVC, MultinomialNB)
- pdfplumber (PDF text extraction)
- joblib (model saving/loading)